Python for Artificial Intelligence

Classification & Tree-Based Models

Week 4, Day 3

THE QUESTION

Will This Ride Get a Quick Pickup?

One question, asked of four different models today.

	Today's setup
The question	Will a driver accept this ride request within 3 minutes?
What we predict	quick_pickup: 1 = Quick Pickup, 0 = Long Wait
Features (3)	distance_km, time_of_day (Morning, Afternoon, Evening, Night), area_type (Commercial, Residential, Airport)
What the models see (6 columns)	distance_km, time_of_day_Evening, time_of_day_Morning, time_of_day_Night, area_type_Commercial, area_type_Residential (after one-hot encoding with drop_first=True)
Data and split	ride_requests.csv: 800 rides to train, 200 to test (test_size=0.2, random_state=42), the same for every model

Today's Project: Will This Ride Get a Quick Pickup?

- Yesterday you predicted a number (fare). Today you predict a category: Quick Pickup or Long Wait
- Hour 1: Classification, Logistic Regression, and K-Nearest Neighbors (KNN)
- Hour 2: Decision Trees and Random Forest
- Hour 3: a proper model report card, comparing all four models on ride_requests.csv

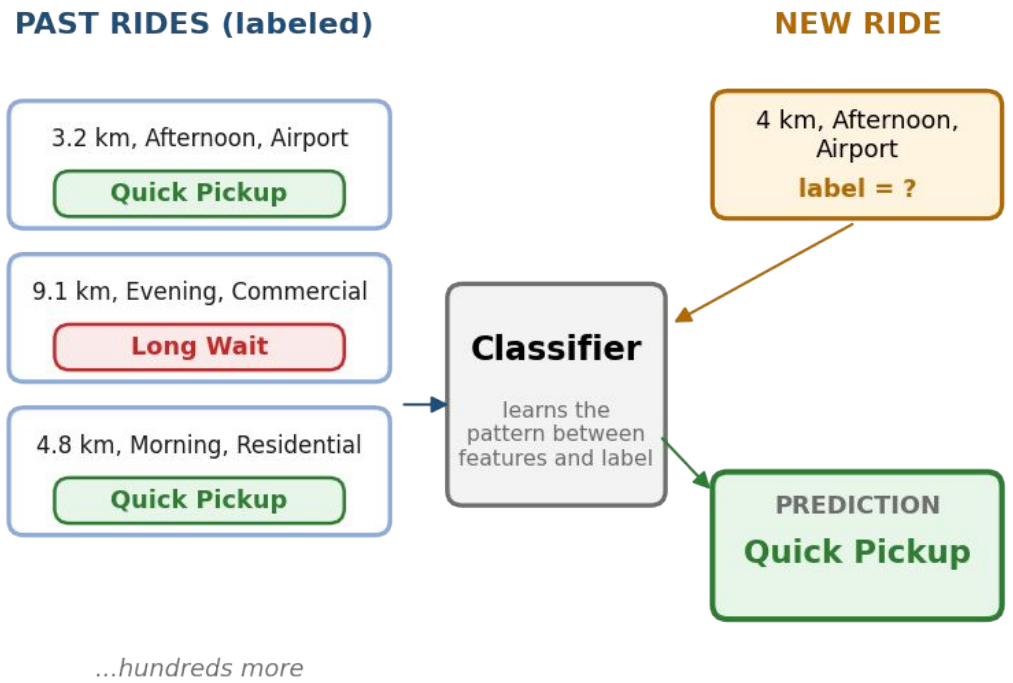
Hour 1

Classification, Logistic Regression & KNN

CONCEPT

What Is Classification?

- Regression (yesterday) predicts a number. Classification predicts a category
- Will this ride get picked up fast? Will this customer churn? Is this email spam? All classification questions
- The simplest case, two possible categories, is called binary classification: here, Quick Pickup vs. Long Wait
- Many classification models output a probability first, then convert it into a category



Classification: features in, a category out

PROJECT

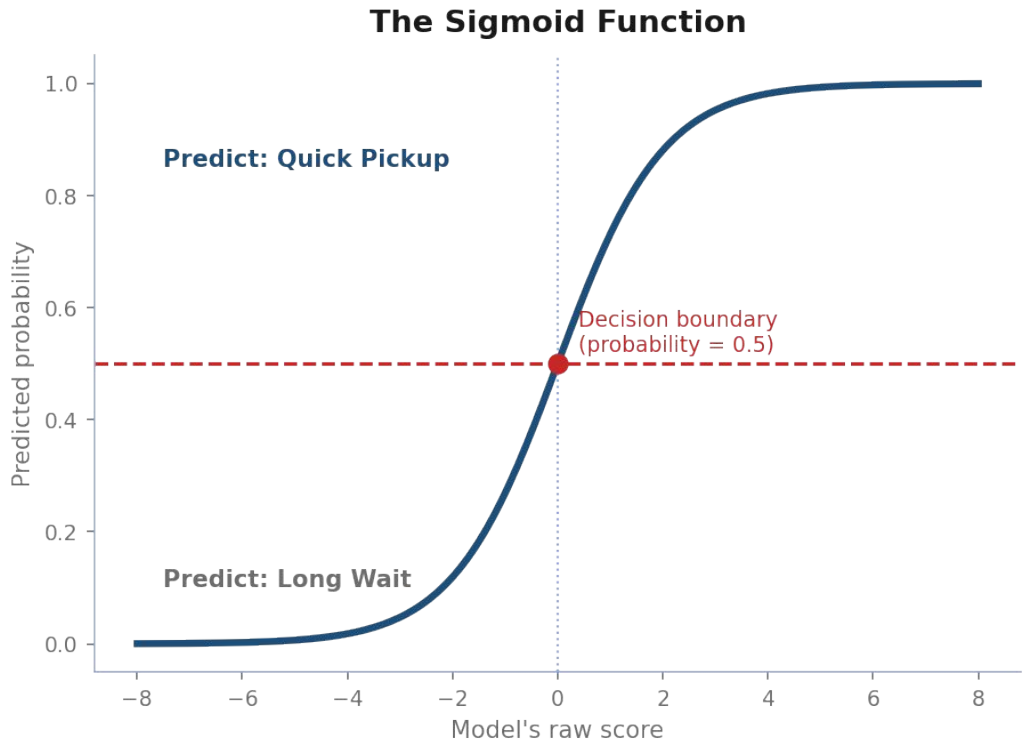
A New Label: quick_pickup

- Same ride_requests.csv from Day 17, one more column in play: quick_pickup (1 = a driver accepted within 3 minutes)
- Features to predict it: distance_km, time_of_day, and area_type
- The real product question: should the app warn a rider "this might take a while" before they book?
- A genuinely useful prediction: it changes what the app tells the rider, in real time

CONCEPT

Logistic Regression

- Despite the name, Logistic Regression is used for classification, not predicting numbers
- It calculates a score from the features, then squashes that score into a probability between 0 and 1
- The squashing function is called the sigmoid: it turns any number into a value between 0 and 1
- If the probability is above 0.5, predict Quick Pickup; otherwise, predict Long Wait



PROFILE

Logistic Regression: At a Glance

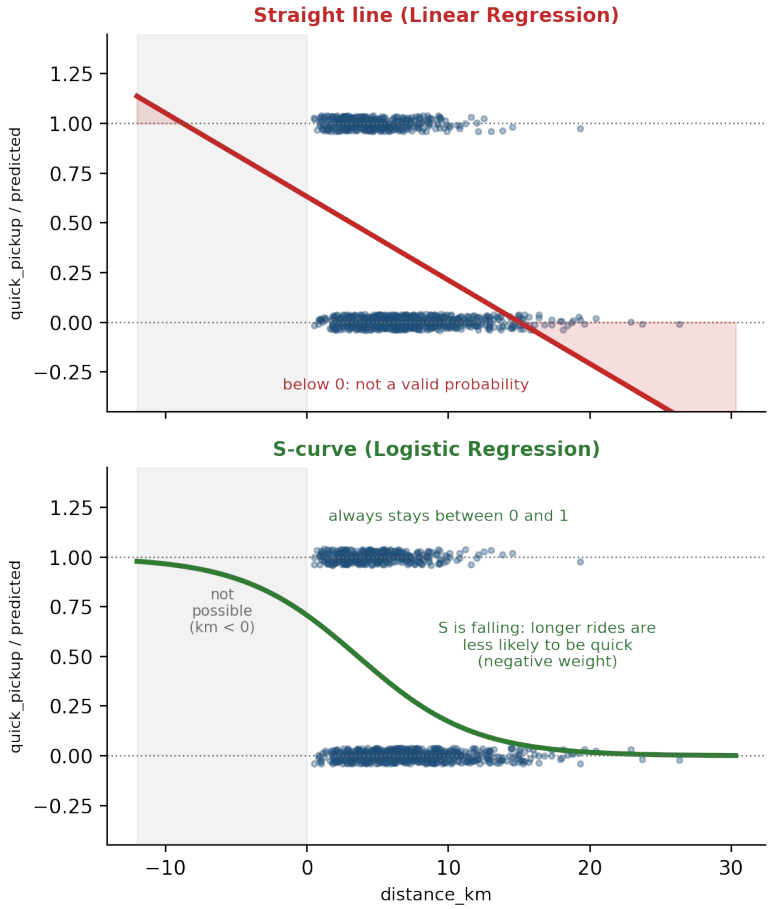
A one-page summary of this algorithm.

Question	Logistic Regression
What does it do?	Combines the features into a score, turns the score into a probability with the sigmoid, then picks a class using a cutoff (usually 0.5)
What is it used for?	Yes/no predictions: spam detection, loan default, disease risk, and here, will this ride get a quick pickup
What is it best at?	A fast, simple baseline. It gives probabilities, not just labels, and its coefficients are readable, so you can explain the result
What are its limits?	It draws a straight-line boundary between classes, so it misses curved or tangled patterns unless you engineer extra features
Use it when	You need probabilities or an explanation, want a quick baseline, or the pattern is roughly linear
Avoid it when	Classes are separated by complex non-linear patterns, or the data is images, audio or raw text

CONCEPT

Why Not Just Use a Straight Line?

- The answer we want is a probability, which must stay between 0 and 1
- A straight line keeps going: far enough along the x-axis it predicts below 0 or above 1, and a negative probability makes no sense
- In the top chart, rides over about 15 km get a negative prediction
- The labels are only 0 or 1, so a line fits them poorly: the dots sit in two flat rows, not along a slope
- The sigmoid bends into an S that flattens near 0 and 1. Here it slopes down because longer rides are less likely to be quick



THE MATH

Logistic Regression: The Math

- The score z is a weighted sum of the features, exactly like Linear Regression
- The sigmoid turns z into a probability p between 0 and 1
- The sign of a weight sets the direction of the S: a positive weight makes it rise, a negative weight flips it so it falls (distance_km has a negative weight)
- Decision rule: predict Quick Pickup when p is at least 0.5, which happens when z is at least 0
- Training minimises log loss: it punishes confident wrong answers heavily

1. Score

$$z = b_0 + b_1x_1 + b_2x_2 + \ \ldots\ + b_nx_n$$

features times their learned weights, plus the intercept

2. Probability (sigmoid)

$$p = \frac{1}{1+e^{-z}}$$

squashes any score into a value between 0 and 1

3. Decision rule

$$\hat{y} = \text{Quick Pickup} \ \text{ if } \ p \geq 0.5$$

otherwise predict Long Wait

What training minimizes (log loss)

$$-\frac{1}{n}\sum[\,y \ln p + (1-y)\ln(1-p)\,]$$

punishes confident wrong answers the most

HOW IT WORKS

Logistic Regression: Step by Step

One ride from our dataset: 4 km, Afternoon, Airport. The model was trained on 800 rides.

Step	What happens	Our ride
Train	Find the weights that make the predicted probabilities match the real labels in the 800 training rides	Intercept 1.3615, distance weight -0.2544
1. Score	Multiply each feature by its weight, add them up and add the intercept	z = 1.3615 + (-0.2544 x 4) = 0.344
2. Probability	Pass the score through the sigmoid to get a value between 0 and 1	p = 1 / (1 + e^-0.344) = 0.585
3. Compare	Check the probability against the cutoff (0.5)	0.585 is above 0.5
4. Predict	Above the cutoff means class 1, otherwise class 0	Quick Pickup

EXAMPLE

Fitting a Logistic Regression Model

The same fit/predict pattern as Linear Regression, now for a category.

```
from sklearn.linear_model import LogisticRegression

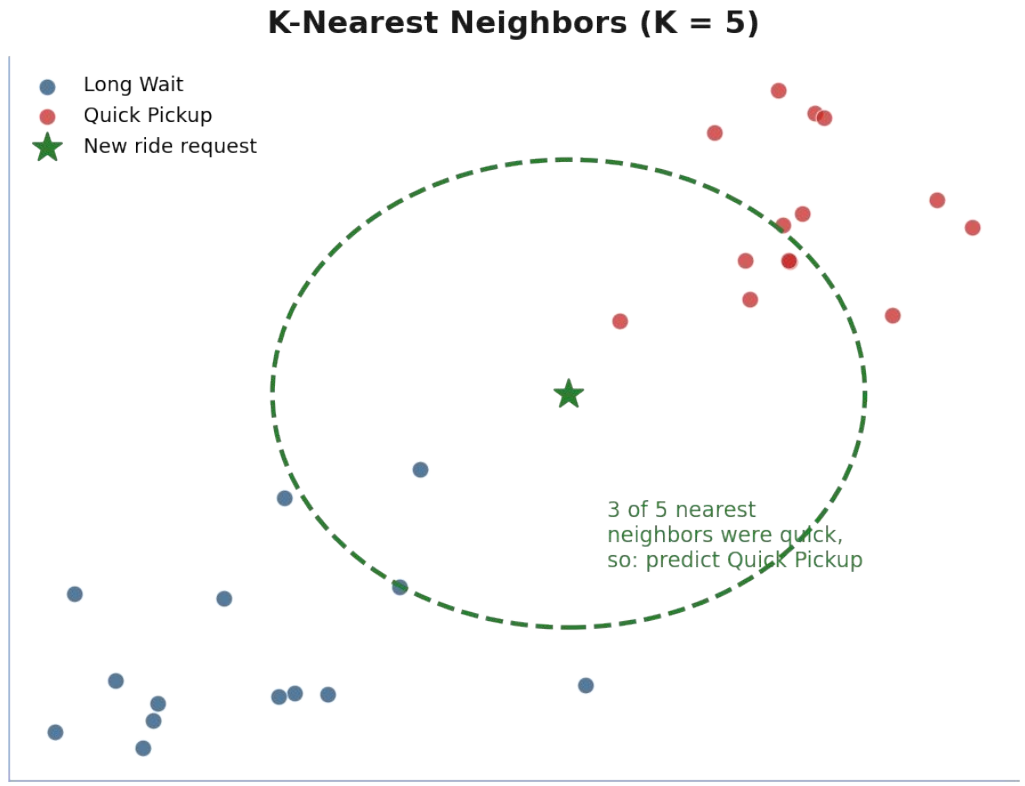
model = LogisticRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)
probabilities = model.predict_proba(X_test)
# probabilities gives [P(Long Wait), P(Quick Pickup)] for each ride
```

CONCEPT

K-Nearest Neighbors (KNN)

- KNN classifies a new ride by looking at its K most similar past rides in the training data
- It takes a majority vote: whichever class is most common among those K neighbors wins
- No formula is learned in advance, the "model" is really just the stored training rides themselves
- Choosing K matters: too small and it's noisy, too large and it blurs real boundaries between classes



PROFILE

K-Nearest Neighbors: At a Glance

A one-page summary of this algorithm.

Question	K-Nearest Neighbors
What does it do?	Finds the K most similar past rides to a new ride and lets them vote: the most common class among them wins
What is it used for?	Recommendations, finding similar items, small classification problems, and spotting odd cases that look unlike their neighbors
What is it best at?	Simple to understand, no training step, and it can follow complex boundaries without assuming any formula
What are its limits?	Slow on big data (it compares against every stored ride), needs features on the same scale, and gets worse with many or irrelevant features
Use it when	The dataset is small to medium, there are only a few meaningful numeric features, and you want a quick no-assumptions baseline
Avoid it when	The dataset is huge, there are many features, or you need very fast predictions on demand

THE MATH

KNN: The Math

- Distance between two rides is the Euclidean distance: the square root of the sum of squared differences
- Example: 4 km Afternoon vs. 9 km Evening gives sqrt(5^2 + 1^2) = 5.1
- Prediction is the most common label among the K closest rides
- Features with big numbers (like km) dominate the distance, so scale features first

1. Distance between two rides (Euclidean)

$$d(a, b) = \sqrt{\sum_i (a_i - b_i)^2}$$

a_i and b_i are the same feature (e.g. distance_km) for ride a and ride b

2. Pick the K closest rides

$$N_K(x) = \{ K \text{ rides with smallest } d(x, \cdot) \}$$

K is chosen by you, e.g. K = 5

3. Vote

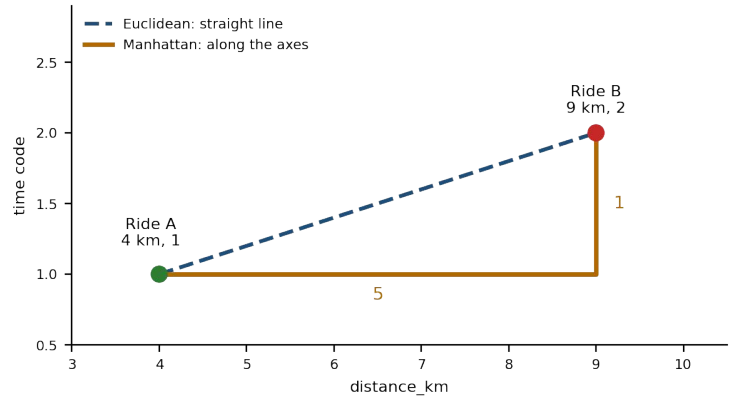
$$\hat{y} = \text{most common label in } N_K(x)$$

share of Quick Pickup neighbors = predicted probability

THE MATH

KNN: Euclidean vs. Manhattan Formulas

- KNN needs a way to measure how far apart two rides are; there is more than one choice
- Euclidean: square each feature difference, add them up, take the square root (the straight-line distance)
- Manhattan: add up the absolute difference in each feature (the distance along the axes, like city blocks)
- Same two rides, different answers: 5.1 with Euclidean, 6 with Manhattan
- Both count a ride as closer when its features are more alike



Euclidean distance

$$d = \sqrt{\sum_i (a_i - b_i)^2}$$

Ride A vs. Ride B: $\sqrt{5^2 + 1^2} = 5.1$

Manhattan distance

$$d = \sum_i |a_i - b_i|$$

Ride A vs. Ride B: $|5| + |1| = 6$

COMPARISON

Euclidean vs. Manhattan: Which to Use

Both work for KNN. The right one depends on your data.

	Euclidean	Manhattan
Idea	Straight-line distance between two points	Total distance walked along each feature, like city blocks
Formula	sqrt(sum of (a - b)^2)	sum of |a - b|
Our example	sqrt(5^2 + 1^2) = 5.1	|5| + |1| = 6
One big gap vs. many small	Gaps of (4, 0, 0, 0) give 4.0, but (1, 1, 1, 1) give only 2.0, so one large gap counts for more	Both give 4: only the total gap matters, not where it comes from
Best when	Few continuous features on similar scales; you want true closeness	Many features, mostly 0/1 columns, grid-like movement such as streets, or you do not want one feature to dominate
In scikit-learn	metric="euclidean" (the default)	metric="manhattan"

HOW IT WORKS

KNN: Step by Step

KNN has no training formula. It keeps the data and does the work at prediction time.

Step	What happens	Our ride
0. Store	"Training" is only storing the 800 labelled rides	Nothing is learned yet
1. Measure	Calculate the distance from the new ride to every stored ride	800 distances computed
2. Pick K	Keep the K closest rides (we use K = 5)	The 5 rides nearest to 4 km, Afternoon, Airport
3. Vote	Count the labels of those K neighbours	4 Quick, 1 Long
4. Predict	The majority label wins	Quick Pickup (4 of 5 = 0.8)

WORKED EXAMPLE

The 5 Nearest Rides

The neighbours KNN found for our ride (4 km, Afternoon, Airport). All five share its time and area, so only distance differs.

Neighbour	distance_km	time_of_day	area_type	Distance to new ride	Label
1	4.0	Afternoon	Airport	0.0	Quick
2	4.1	Afternoon	Airport	0.1	Quick
3	4.2	Afternoon	Airport	0.2	Quick
4	3.8	Afternoon	Airport	0.2	Quick
5	3.7	Afternoon	Airport	0.3	Long

EXAMPLE

Fitting a KNN Classifier

n_neighbors sets K, the number of past rides that get a vote.

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
# try a few different values of n_neighbors and compare results
```

QUICK CHECK

Quick Check

A Logistic Regression model outputs a probability of 0.37 for Quick Pickup on a given ride. What does it predict, and why?

ANSWER

Quick Check: Answer

```
# It predicts Long Wait.
#
# The default decision boundary is 0.5. Since 0.37 is below
# 0.5, the model is more confident in Long Wait than in
# Quick Pickup, so Long Wait is the predicted category.
```

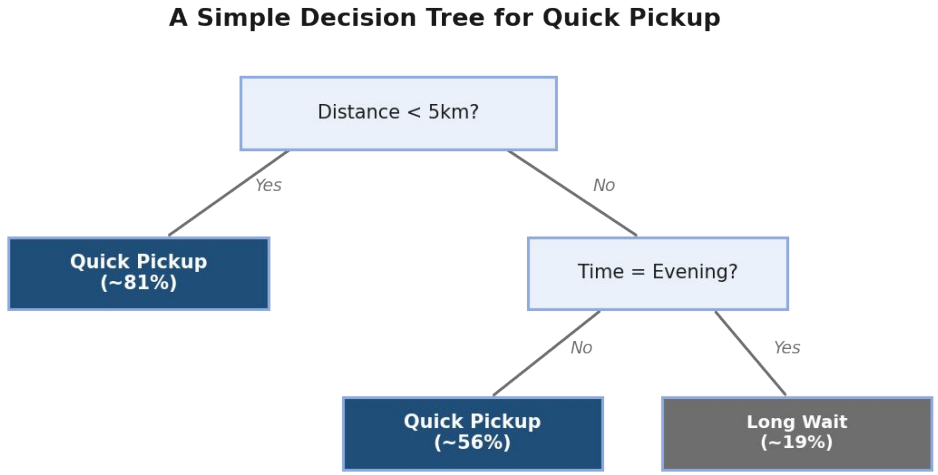
Hour 2

Decision Trees & Random Forest

CONCEPT

Decision Trees

- A Decision Tree classifies by asking a series of yes/no questions, like a flowchart
- Each question (a "split") is chosen to separate Quick Pickup from Long Wait as cleanly as possible
- You follow the branches down until you reach a leaf, which holds the final prediction
- Trees are easy to read and explain, you can literally show a rider the exact reasoning behind a wait-time warning



PROFILE

Decision Tree: At a Glance

A one-page summary of this algorithm.

Question	Decision Tree
What does it do?	Asks a chain of yes/no questions about the features. Each split separates the classes as cleanly as possible until a leaf gives the answer
What is it used for?	Decisions that must be explained: credit approval, medical triage rules, customer rules, and here, warning a rider about a long wait
What is it best at?	Easy to read and show to others. Handles numbers and categories, needs no scaling, and captures non-linear patterns and feature interactions
What are its limits?	Overfits easily if it grows deep, and a small change in the data can produce a very different tree
Use it when	You need clear rules you can explain, your features are a mix of numbers and categories, or you want a quick non-linear baseline
Avoid it when	You need the best possible accuracy (a forest does better), or the data is small and noisy and you cannot limit the depth

THE MATH

Decision Tree: The Math

- Gini impurity measures how mixed a group is: 0 is pure, 0.5 is a 50/50 mix
- After a split, take the weighted average of the two groups' Gini
- Information gain is the Gini before the split minus the weighted Gini after
- The tree picks the split with the largest gain

1. Impurity of a node (Gini)

$$G = 1 - p_{\text{quick}}^2 - p_{\text{long}}^2$$

0 = pure (all one class), 0.5 = a perfect 50/50 mix

2. Impurity after a split

$$G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R$$

weighted by how many rides go left and right

3. Gain: pick the split with the biggest drop

$$\text{Gain} = G_{\text{parent}} - G_{\text{split}}$$

tried for every feature and threshold, best one wins

HOW IT WORKS

Decision Tree: Step by Step

Building the tree, then using it on our ride (4 km, Afternoon, Airport).

Step	What happens	Our ride
1. Try splits	For every feature and threshold, test how well it separates Quick from Long	e.g. distance_km at 5.85
2. Pick the best	Choose the split that lowers impurity (Gini) the most	distance_km <= 5.85 wins the root
3. Repeat	Do the same inside each branch until max_depth (4) or the groups are pure	Tree grows to 4 levels
4. Predict	Follow the yes/no answers from the root down to a leaf	4 km: distance <= 5.85 yes, <= 2.95 no, Commercial no, Residential no
5. Answer	The leaf's majority class is the prediction	Leaf is 57.5% quick, so Quick Pickup

WORKED EXAMPLE

Worked Example: Choosing the Root Split

Splitting 800 training rides on distance_km <= 5.85.

Group	Rides	% quick	Gini
Root (before split)	800	36.4%	0.463
Left: distance <= 5.85	392	51.5%	0.500
Right: distance > 5.85	408	21.8%	0.341
Weighted Gini after split	800		0.419
Gain (0.463 - 0.419)			0.044

CONCEPT

Overfitting in Trees, and Random Forest

- A tree with no limits keeps splitting until every training ride is perfectly classified, that's overfitting
- max_depth limits how many questions deep the tree can go, trading some training accuracy for better generalization
- Random Forest builds many different trees on random subsets of the rides, then lets them vote together
- A forest of imperfect trees usually generalizes better than any single, deep tree

PROFILE

Random Forest: At a Glance

A one-page summary of this algorithm.

Question	Random Forest
What does it do?	Builds many decision trees on random samples of the rides and features, then lets all the trees vote
What is it used for?	Fraud detection, customer churn, credit scoring, and tabular business data in general
What is it best at?	Strong accuracy with little tuning. It overfits far less than a single tree and can show which features mattered most
What are its limits?	Hard to explain (hundreds of trees), larger and slower than one tree, and not suited to images, audio or raw text
Use it when	You want a strong accuracy baseline on tabular data and a full explanation is not critical
Avoid it when	You need a simple set of rules to explain, a very small fast model, or the data is images, audio or text

THE MATH

Random Forest: The Math

- Each tree trains on a bootstrap sample: n rides drawn with replacement
- The forest prediction is the majority vote of all the trees
- The predicted probability is simply the share of trees voting for the class
- Averaging many different trees cancels out their individual mistakes

1. Each tree gets a random sample of rides

$$D_b = \text{random sample of the rides}$$

drawn with replacement, so every tree sees slightly different data

2. Every tree votes

$$\hat{y} = \text{majority}\{\, h_1(x),\, h_2(x),\, \ldots,\, h_B(x) \,\}$$

$h_b(x)$ is tree b's prediction, B = number of trees (100)

3. Probability = share of votes

$$P(\text{Quick}) = \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}[h_b(x) = \text{Quick}]$$

e.g. 92 of 100 trees vote Quick Pickup, so 0.92

HOW IT WORKS

Random Forest: Step by Step

Many different trees, one vote. Our ride: 4 km, Afternoon, Airport, with 100 trees.

Step	What happens	Our ride
1. Resample	Give each tree its own random sample of the training rides (bootstrap)	100 slightly different datasets
2. Randomise	At each split, a tree may only consider a random subset of features	Trees end up different from each other
3. Grow	Train one decision tree on each sample	100 trees
4. Vote	Every tree predicts a class for the new ride	Trees 1-7: Q, Q, Q, Q, Q, L, L
5. Combine	The majority class wins; the share of votes is the probability	92 of 100 say Quick, probability 0.92

EXAMPLE

Fitting a Decision Tree and a Random Forest

Both follow the same scikit-learn pattern as every other model today.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

tree = DecisionTreeClassifier(max_depth=4, random_state=42)
tree.fit(X_train, y_train)

forest = RandomForestClassifier(n_estimators=100, random_state=42)
forest.fit(X_train, y_train)
```

QUICK CHECK

Quick Check

Why does a Random Forest usually perform better than a single, very deep Decision Tree?

ANSWER

Quick Check: Answer

```
# A single deep tree tends to overfit: it memorizes the exact
# training rides, noise included.
#
# A Random Forest trains many trees on different random
# subsets of the rides, so each tree's mistakes are a little
# different. Averaging their votes cancels out much of that
# noise, producing a more reliable prediction overall.
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

Hour 3

Building the Model Report Card

CONCEPT

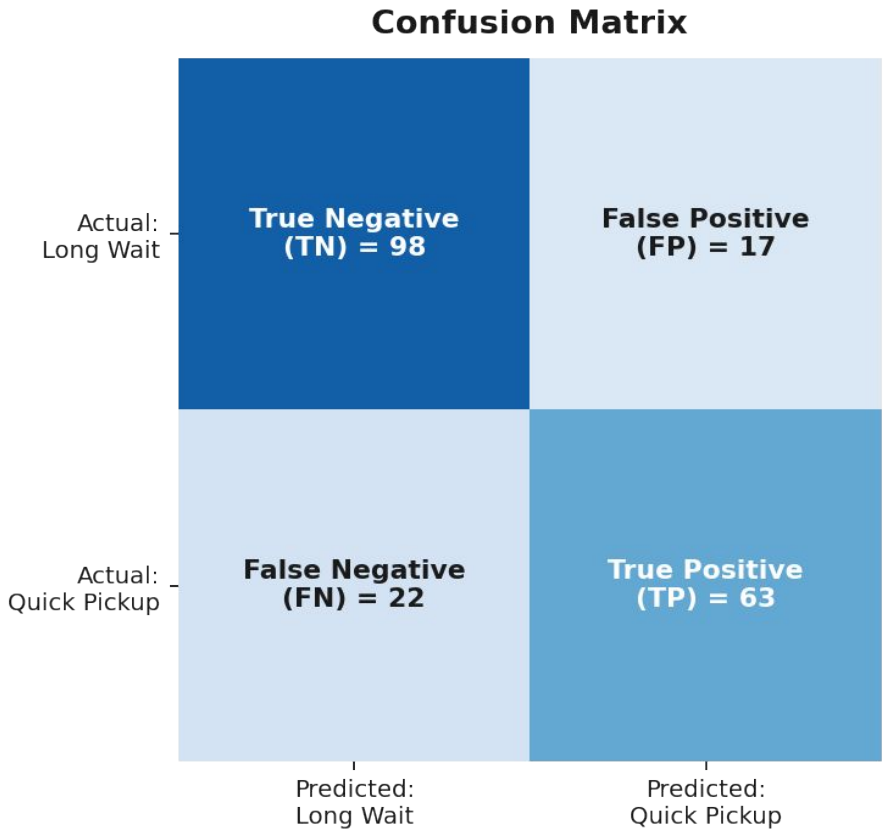
Why Accuracy Isn't Enough

- Accuracy: the percentage of predictions the model got right overall
- Problem: if 80% of rides already get quick pickups, a model that always predicts "Quick Pickup" scores 80% accuracy, and warns nobody
- This is especially dangerous with imbalanced classes, where one category is far more common than the other
- A proper evaluation needs to show how the model performs on each class separately

CONCEPT

The Confusion Matrix

- A confusion matrix breaks every prediction into four buckets: correct and incorrect, for each class
- True Positive (TP): predicted Quick Pickup, actually quick. True Negative (TN): predicted Long Wait, correct
- False Positive (FP): told the rider "quick," but it wasn't. False Negative (FN): warned "long wait," but it was quick
- Every other classification metric is built from these four numbers



THE MATH

Accuracy, Precision, Recall & F1: The Formulas

- All four come from the four confusion-matrix counts: TP, TN, FP, FN
- Numbers shown use the matrix on the previous slide: TP 63, TN 98, FP 17, FN 22
- Accuracy counts every correct prediction; precision and recall look only at the Quick Pickup class
- Precision divides by everything we called Quick (TP + FP); recall divides by everything that was really Quick (TP + FN)
- F1 is the harmonic mean of precision and recall, so one low value pulls it down hard

1. Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

all correct / all rides: 161 / 200 = 0.805

2. Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

of rides we called Quick: 63 / 80 = 0.79

3. Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

of rides that were Quick: 63 / 85 = 0.74

4. F1 score

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

balance of the two: 0.76

REFERENCE

Precision, Recall & F1

Three ways to summarize the confusion matrix, each emphasizing a different kind of mistake.

Metric	Formula (in words)	When It Matters Most
Precision	Of every ride predicted Quick Pickup, how many actually were	False positives annoy riders: promised fast, left waiting
Recall	Of every ride that was actually quick, how many did we catch	False negatives are a missed chance to reassure a rider
F1 Score	The balance between precision and recall	You need one number that penalizes ignoring either side

COMPARISON

Accuracy vs. Precision

Both measure correctness, but they answer different questions.

	Accuracy	Precision
Question it answers	Out of all rides, how many predictions were right?	When the model says Quick Pickup, how often is it right?
Formula	(TP + TN) / (TP + TN + FP + FN)	TP / (TP + FP)
What it looks at	Every ride, both classes	Only the rides predicted as Quick Pickup
It can be fooled by	Imbalanced classes: always guessing the common class scores high	Predicting Quick rarely: a few safe guesses give a high score
Ride example	Always say Long Wait: accuracy 0.66 on our test set, yet it never warns anyone about a quick pickup	Say Quick for one ride and be right: precision 1.00, yet 67 quick rides were missed (recall near 0)
Use it when	Classes are balanced and every mistake costs about the same	A false alarm is costly: promising a quick pickup that does not happen

EXAMPLE

Getting a Full Report

classification_report prints precision, recall, and F1 for every class at once.

```
from sklearn.metrics import classification_report, confusion_matrix

predictions = model.predict(X_test)

print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
```

EXAMPLE

Comparing All Four Models in One Table

Loop over the models, fit each one, collect its metrics, then print one DataFrame.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN (K=5)": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(max_depth=4, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
}
rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({"Model": name, "Accuracy": accuracy_score(y_test, pred),
                 "Precision": precision_score(y_test, pred),
                 "Recall": recall_score(y_test, pred), "F1": f1_score(y_test, pred)})
scores = pd.DataFrame(rows).set_index("Model")
print(scores.round(2))
```

COMPARISON

Model Report Card: All Four Models

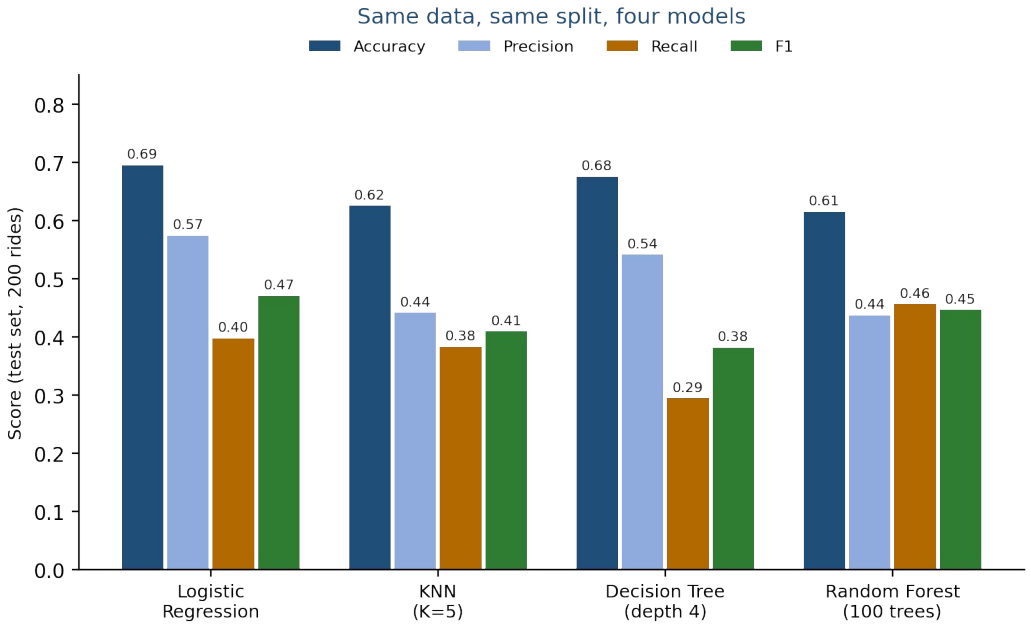
Predicting Quick Pickup on the same 200 held-out rides (random_state=42). Class 1 = Quick Pickup.

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.70	0.57	0.40	0.47
KNN (K=5)	0.62	0.44	0.38	0.41
Decision Tree (depth 4)	0.68	0.54	0.29	0.38
Random Forest (100 trees)	0.62	0.44	0.46	0.45

COMPARISON

Reading the Report Card

- The simplest model, Logistic Regression, has the best accuracy and F1. A fancier model is not automatically better
- Random Forest catches the most Quick Pickups (recall 0.46) but raises more false alarms (precision 0.44)
- The Decision Tree is cautious: decent precision, but it misses 7 in 10 quick rides (recall 0.29)
- Only 34% of test rides are quick, so a model that always says Long Wait would score 0.66 accuracy. Compare against that
- Which to ship depends on the cost of a mistake: promising a quick pickup that does not happen, or failing to warn about a wait that does



COMPARISON

Which Metric Should You Use?

No metric is best everywhere. Choose by what a mistake costs.

Metric	Use it when	In our ride app	Winner on our models
Accuracy	Classes are balanced and both mistakes cost about the same	Weak choice: only 34% of rides are quick, so always guessing Long Wait scores 0.66	Logistic Regression (0.70)
Precision	False alarms hurt most	Promising a quick pickup that does not happen	Logistic Regression (0.57)
Recall	Missing a real case hurts most	Failing to reassure a rider who would have been picked up fast	Random Forest (0.46)
F1	You want one number balancing precision and recall, especially with imbalanced classes	Best single score when neither mistake clearly dominates	Logistic Regression (0.47)

CONCEPT

Choosing a Metric: Habits and a Rule of Thumb

- Decide the metric before you look at the results, based on what a mistake costs. Otherwise people pick the metric that makes their favourite model look best
- Always compare against a baseline, such as always predicting the most common class, to check the model is better than guessing
- If you can only look at one number, use F1. If the app must avoid broken promises, use precision. If it must catch every quick ride, use recall

PRACTICE

Practice

On ride_requests.csv, train a Logistic Regression model, a KNN model (try K=5), and a Decision Tree (max_depth=4) to predict quick_pickup from distance_km, time_of_day, and area_type. Print a classification_report for each, and build a one-page report card: which model has the best recall for Quick Pickup, and which would you actually ship in the app?

Key Takeaways & Coming Up

- Classification predicts a category; Logistic Regression, KNN, and Decision Trees each reach that prediction differently
- Random Forest trades a single tree's overfitting risk for the stability of many trees voting together
- Accuracy alone can be misleading, especially with imbalanced classes, precision, recall, and F1 tell the fuller story
- Your project now predicts both a fare (Day 17) and a quick-pickup probability (today), with a report card to back the choice of model
- Coming up next: neural networks, conceptually, starting with how a single neuron would score one ride request by hand